EVALUASI TENGAH SEMESTER (ETS)

Algoritma dan Pemrograman

Program Studi Teknik Instrumentasi
Semester Genap 2025/2026

PROJECT BASED LEARNING (PBL)

Pengembangan Sistem Instrumentasi
Pengukuran dan Kontrol Berbasis Rust

Studi Kasus:

Monitoring Kualitas Udara Real-Time

Dosen Pengampu : Ahmad Radhy, S.Si., M.Si.
Durasi Perngerjaan : 1 Pekan
Bahasa Pemrograman : Rust
Metode Penilaian : Project + Ujian Lisan

No.	Nama	NRP
Mahasiswa 1	Arief Rahman	2042251034
Mahasiswa 2	Muhammad Alfath Muslim Anatin	2042251120

Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember
Surabaya, 2026

1. Pendahuluan

1.1. Latar Belakang

Kualitas udara merupakan salah satu faktor lingkungan yang berdampak langsung terhadap kesehatan manusia dan ekosistem. Peningkatan aktivitas industri, transportasi, dan pembakaran bahan bakar fosil menyebabkan konsentrasi polutan udara seperti partikel halus (PM2.5/PM10), karbon monoksida (CO), nitrogen dioksida (NO_2), dan ozon (O_3) terus meningkat, terutama di kawasan perkotaan dan area industri.

Pemantauan kualitas udara secara konvensional umumnya dilakukan menggunakan peralatan stasioner berukuran besar dengan biaya operasional yang tinggi dan jangkauan pengukuran yang terbatas. Akibatnya, data yang diperoleh seringkali tidak merepresentasikan kondisi riil di lapangan secara menyeluruh dan berkelanjutan.

Perkembangan teknologi sensor dan sistem tertanam (*embedded system*) membuka peluang untuk merancang sistem monitoring yang lebih terjangkau, portabel, dan mampu bekerja secara *real-time*. Namun demikian, sistem tertanam yang menangani data sensor secara simultan rentan terhadap permasalahan seperti akses memori yang tidak aman, kondisi *race condition*, dan kegagalan sistem akibat kesalahan manajemen sumber daya.

Bahasa pemrograman Rust hadir sebagai solusi atas permasalahan tersebut. Rust dirancang dengan konsep *ownership* dan *borrowing* yang menjamin keamanan memori (*memory safety*) secara statis pada saat kompilasi, tanpa memerlukan *garbage collector*. Hal ini menjadikan Rust sangat sesuai untuk sistem tertanam yang membutuhkan efisiensi tinggi sekaligus keandalan dalam mengelola konkurensi antar proses pembacaan sensor, kalkulasi AQI, dan pengiriman data secara nirkabel.

Berdasarkan latar belakang tersebut, studi kasus ini merancang sebuah sistem monitoring kualitas udara lingkungan berbasis Rust yang mampu membaca data dari berbagai sensor (PM2.5/PM10, CO, NO_2, suhu, dan kelembapan), melakukan kalibrasi dan perhitungan *Air Quality Index* (AQI) secara otomatis, serta menampilkan dan mengirimkan hasil pemantauan secara *real-time* sebagai bentuk penerapan algoritma dan pemrograman sistem instrumentasi modern.

1.2. Tujuan

Tujuan dari studi kasus ini adalah:

1. Merancang sistem monitoring kualitas udara lingkungan berbasis bahasa pemrograman Rust pada platform tertanam.
2. Mengimplementasikan algoritma kalibrasi sensor dan perhitungan *Air Quality Index* (AQI) secara real-time.
3. Menerapkan konsep *memory safety* dan konkurensi Rust dalam pengelolaan data multi-sensor secara andal.
4. Mengirimkan dan menampilkan hasil pemantauan kualitas udara secara otomatis melalui antarmuka nirkabel dan display lokal.

1.3. Ruang Lingkup

Batasan permasalahan pada studi kasus ini adalah sebagai berikut:

1. Sensor yang digunakan terbatas pada sensor partikulat (PM2.5/PM10), sensor gas (CO dan NO_2), serta sensor suhu dan kelembapan.
2. Rentang pengukuran konsentrasi gas dibatasi pada 0–2000 ppm untuk CO dan 0–10 ppm untuk NO_2, sesuai kapasitas sensor yang digunakan.
3. Metode kalibrasi yang digunakan adalah interpolasi polinomial orde dua berdasarkan koefisien kalibrasi empiris masing-masing sensor.
4. Perhitungan AQI mengacu pada standar breakpoint EPA (*Environmental Protection Agency*) dengan kategori: Baik, Sedang, Tidak Sehat, dan Berbahaya.
5. Metode kontrol yang diterapkan adalah kontrol ambang batas (*threshold control*) – sistem hanya mengaktifkan alarm apabila nilai AQI melebihi 150.
6. Pengiriman data dilakukan secara satu arah dari node sensor ke server, tanpa mekanisme kendali balik (*feedback control*) dari server ke aktuator.
7. Sistem diimplementasikan menggunakan bahasa pemrograman Rust dan dibatasi pada lingkungan pengujian di dalam ruangan (*indoor environment*).

2. Studi Kasus Instrumentasi

2.1. Deskripsi Sistem

Sistem yang dikaji adalah Sistem Monitoring Kualitas Udara Lingkungan (Environmental Air Quality Monitoring System). Sistem ini dirancang untuk memantau secara real-time parameter-parameter kualitas udara di lingkungan perkotaan maupun industri, meliputi konsentrasi gas berbahaya, partikel debu, suhu, dan kelembapan.

2.2. Prinsip Kerja

Sistem bekerja berdasarkan prinsip deteksi elektrokimia, optik, dan resistif dari masing-masing sensor. Setiap sensor menghasilkan tegangan atau arus keluaran yang sebanding dengan konsentrasi zat yang diukur.

Persamaan kalibrasi sensor gas (elektrokimia):

$$C = \frac{V_{out} - V_{offset}}{S_f} \tag{1}$$

di mana C adalah konsentrasi gas (ppm/ppb), V_{out} adalah tegangan keluaran sensor (mV), V_{offset} adalah tegangan offset saat tidak ada gas, dan S_f adalah sensitivitas sensor (mV/ppm).

Persamaan sensor partikulat (optik hamburan cahaya):

$$PM = a_0 + a_1 \cdot R_{dust} + a_2 \cdot R_{dust}^2 \tag{2}$$

di mana PM adalah konsentrasi partikel (μg/m^3), R_{dust} adalah rasio keluaran tegangan sensor optik, dan a_0, a_1, a_2 adalah koefisien kalibrasi empiris.

Perhitungan *Air Quality Index* (AQI):

$$AQI = \frac{I_{high} - I_{low}}{C_{high} - C_{low}} \cdot (C_p - C_{low}) + I_{low} \tag{3}$$

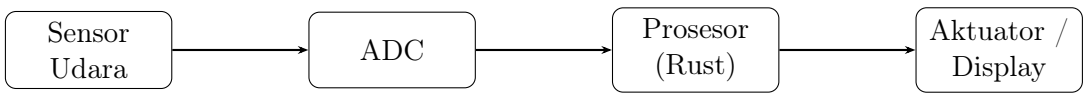
di mana C_p adalah konsentrasi polutan terukur, C_{low} dan C_{high} adalah batas bawah dan atas kategori konsentrasi, serta I_{low} dan I_{high} adalah nilai AQI batas bawah dan atas kategori yang bersesuaian.

Contoh persamaan kalibrasi sensor:

$$T = a_0 + a_1 \cdot V + a_2 \cdot V^2 \tag{4}$$

di mana T adalah suhu (°C), V adalah tegangan keluaran (mV), dan a_0, a_1, a_2 adalah koefisien kalibrasi.

2.3. Blok Diagram Sistem



Gambar 1: Blok Diagram Sistem Monitoring Kualitas Udara

2.4. Spesifikasi Teknis

Tabel 1: Spesifikasi Teknis Sistem Monitoring Kualitas Udara

Komponen	**Spesifikasi**
Sensor Partikulat (PM2.5/PM10)	Rentang 0 – 500 μg/m^3, antarmuka UART
Sensor Gas (CO, NO_2, O_3)	Rentang 0 – 2000 ppm, keluaran analog
Sensor Suhu & Kelembapan	-40 s.d. $+80\,^\circ$C, 0 – 100% RH
ADC	Resolusi 16-bit, antarmuka I^{2}C
Prosesor (Rust)	Mikrokontroler/SBC, bahasa Rust
Komunikasi	UART, I^{2}C, LoRa / WiFi (MQTT)
Keluaran	LCD, buzzer alarm, penyimpanan SD Card
Catu Daya	5V DC, $\leq$ 2.5 W, baterai cadangan
Lingkungan Operasi	-10 s.d. $+60\,^\circ$C, proteksi IP54

3. Algoritma dan Flowchart

3.1. Deskripsi Algoritma

Sistem monitoring kualitas udara bekerja melalui lima tahap utama yang berjalan secara berulang (*loop*).

Tahap 1 — Inisialisasi. Saat sistem pertama dinyalakan, program Rust melakukan konfigurasi awal: membuka koneksi serial ke sensor, mengatur pin GPIO, serta menginisialisasi ADC melalui bus I^2C. Semua variabel global dan buffer data dikosongkan sebelum proses pembacaan dimulai.

Tahap 2 — Pembacaan Sensor. Program membaca data dari masing-masing sensor secara bergantian. Sensor partikulat mengirim data via UART, sedangkan sensor gas dan suhu/kelembapan dibaca melalui ADC. Setiap pembacaan diulang sebanyak tiga kali dan dirata-rata untuk mengurangi derau.

Tahap 3 — Kalibrasi dan Perhitungan AQI. Nilai mentah (*raw*) dari sensor dikonversi menggunakan persamaan kalibrasi masing-masing sensor. Setelah didapat konsentrasi sebenarnya, nilai AQI dihitung menggunakan rumus interpolasi linear berdasarkan tabel *breakpoint* standar EPA/BMKG.

Tahap 4 — Evaluasi Ambang Batas. Nilai AQI yang diperoleh dibandingkan dengan ambang batas yang telah ditetapkan. Jika AQI melebihi 150 (kategori tidak sehat), sistem mengaktifkan alarm buzzer dan menandai status peringatan pada paket data yang akan dikirim.

Tahap 5 — Pengiriman Data dan Logging. Data hasil pengukuran beserta status AQI dikirim ke server melalui LoRa atau WiFi menggunakan protokol MQTT. Secara bersamaan, data juga disimpan ke SD Card sebagai log lokal. Setelah selesai, sistem menunggu interval waktu tertentu (60 detik) sebelum kembali ke Tahap 2.

3.2. Pseudocode

Algorithm 3.1 Algoritma Monitoring Kualitas Udara (Rust)

MULAI
  Inisialisasi sensor dan parameter sistem
loop ▷ ULANGI
  Baca data sensor → `nilai_raw`
  Kalibrasi `nilai_raw` → `nilai_fisik`

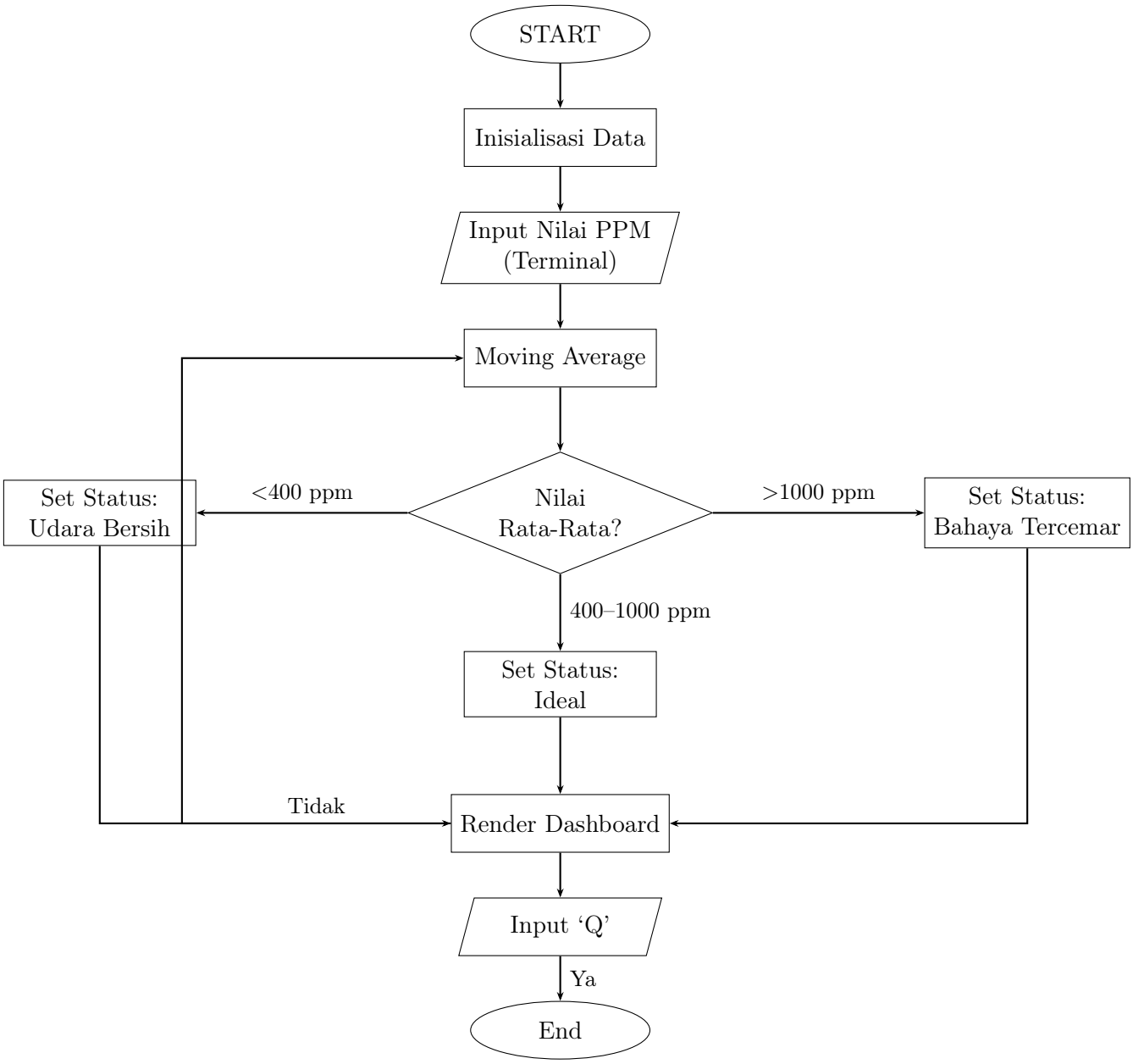
$$C = \frac{V_{out} - V_{offset}}{S_f}, \quad PM = a_0 + a_1 \cdot R_{dust} + a_2 \cdot R_{dust}^2$$

  Hitung AQI dari `nilai_fisik`

$$AQI = \frac{I_{high} - I_{low}}{C_{high} - C_{low}} \cdot (C_p - C_{low}) + I_{low}$$

  if $AQI > 150$ **then**
    Aktifkan alarm → `status = "TIDAK SEHAT"`
  else
    `status = "AMAN"`
  end if
  Kirim perintah ke aktuator
  Tampilkan hasil ke layar / log
  Tunggu interval waktu (60 detik)
end loop ▷ SAMPAI sistem dihentikan
SELESAI

3.3. Flowchart



Gambar 2: Flowchart Sistem Monitoring Kualitas Udara

4. Penjelasan Program Rust

Program yang dibuat merupakan *dashboard* interaktif berbasis terminal untuk monitoring kualitas udara secara *real-time*. Program ditulis seluruhnya dalam satu file `main.rs` menggunakan bahasa pemrograman Rust dengan memanfaatkan dua *crate* eksternal utama, yaitu `crossterm` untuk pengelolaan input/output terminal lintas platform, dan `ratatui` untuk pembangunan antarmuka pengguna berbasis teks (TUI – *Text User Interface*).

4.1. Struktur Proyek

Proyek dikonfigurasi sebagai *binary crate* standar Rust dengan struktur sebagai berikut:

```
monitoring_udara/
 Cargo.toml       <- konfigurasi proyek dan dependensi
 src/ main.rs      <- seluruh source code program
```

4.2. Cargo.toml

```toml
[package]
name    = "monitoring_udara"
version = "0.1.0"
edition = "2024"

[dependencies]
crossterm = "0.27"
ratatui   = "0.26"
```

Listing 1: Cargo.toml

Keterangan dependensi:

- **crossterm**: library lintas platform untuk mengontrol terminal – mengaktifkan *raw mode*, menangkap event keyboard, dan memanipulasi layar terminal.
- **ratatui**: library TUI (Text User Interface) untuk membangun widget visual di dalam terminal, seperti gauge, paragraf, dan layout bertingkat.

4.3. Deklarasi *Use* dan Impor Modul

```rust
use std::io;
use std::time::Duration;
use crossterm::{
    event::{self, DisableMouseCapture, EnableMouseCapture, Event,
   KeyCode},
    execute,
    terminal::{disable_raw_mode, enable_raw_mode,
               EnterAlternateScreen, LeaveAlternateScreen},
};
use ratatui::{
    backend::CrosstermBackend,
    layout::{Constraint, Direction, Layout},
```

```
    style::{Color, Modifier, Style},
    widgets::{Block, Borders, Gauge, Paragraph, Wrap},
    Terminal,
};
```

Listing 2: Impor library dan modul

Bagian ini merupakan deklarasi semua modul dan tipe yang akan digunakan sepanjang program. Penjelasan tiap baris:

- `use std::io` – mengimpor modul I/O standar Rust. Modul ini dibutuhkan karena fungsi `main()` mengembalikan tipe `Result<(), io::Error>` dan operasi penulisan ke stdout menggunakan `io::stdout()`.
- `use std::time::Duration` – mengimpor tipe `Duration` untuk mendefinisikan interval waktu. Digunakan pada fungsi `event::poll(Duration::from_millis(100))` agar sistem tidak memblokir loop utama lebih dari 100 ms saat menunggu input.
- `crossterm::event` – kelompok impor untuk penanganan event terminal:
  - `DisableMouseCapture` / `EnableMouseCapture` – mengaktifkan/menonaktifkan penangkapan event mouse.
  - `Event` – enum yang merepresentasikan event terminal (KeyEvent, MouseEvent, dsb.).
  - `KeyCode` – enum kode tombol keyboard (`KeyCode::Up`, `KeyCode::Down`, `KeyCode::Char('q')`, dsb.).
- `crossterm::execute!` – macro untuk mengeksekusi perintah terminal secara berurutan ke sebuah *writer* (misalnya stdout).
- `enable_raw_mode` / `disable_raw_mode` – mengaktifkan dan menonaktifkan *raw mode* terminal. Dalam raw mode, karakter yang diketik tidak secara otomatis dicetak ke layar dan tidak ada pemrosesan baris (enter tidak diperlukan), sehingga program dapat membaca input karakter per karakter secara langsung.
- `EnterAlternateScreen` / `LeaveAlternateScreen` – perintah untuk masuk ke layar alternatif terminal (buffer terpisah dari sesi shell normal), sehingga konten terminal asli tidak tertimpa selama program berjalan.
- `ratatui::backend::CrosstermBackend` – implementasi backend ratatui yang menggunakan crossterm sebagai lapisan komunikasi dengan terminal.
- `ratatui::layout::{Constraint, Direction, Layout}` – sistem tata letak ratatui untuk membagi area layar menjadi beberapa panel (chunk) secara vertikal maupun horizontal.
- `ratatui::style::{Color, Modifier, Style}` – tipe untuk mendefinisikan gaya visual seperti warna teks, warna latar, dan modifier seperti `BOLD` (tebal).
- `ratatui::widgets::{Block, Borders, Gauge, Paragraph, Wrap}` – widget-widget yang dirender ke layar terminal:

  - `Block` – kotak pembungkus dengan judul dan border.
  - `Borders` – konstanta untuk mengatur sisi border.
  - `Gauge` – widget *progress bar* untuk menampilkan nilai numerik secara visual.
  - `Paragraph` – multiline widget teks.
  - `Wrap` – konfigurasi pemenggalan baris pada `Paragraph`.

- `ratatui::Terminal` – struct utama yang mengelola siklus render ke backend terminal.

4.4. Struct Sensor - Objek Data Mentah

```
#[allow(dead_code)]
struct Sensor {
    nilai_raw: f32,
}
```

Listing 3: Struct Sensor

`Struct Sensor` mendefinisikan tipe data yang merepresentasikan objek sensor fisik. Field `nilai_raw` bertipe `f32` (bilangan *floating-point* 32-bit) menyimpan nilai pembacaan mentah (*raw*) dari sensor sebelum diproses lebih lanjut.

Atribut `#[allow(dead_code)]` adalah sebuah *compiler attribute* yang memberitahu kompiler Rust untuk tidak menampilkan peringatan (*warning*) `unused field` pada field `nilai_raw`. Hal ini dilakukan karena field tersebut diisi saat pembuatan objek namun hanya diakses sekali via `_sensor_obj.nilai_raw` untuk keperluan tampilan log, sehingga kompiler menganggapnya tidak digunakan secara aktif. Struct ini berfungsi sebagai wujud nyata dari prinsip OOP *encapsulation*, di mana data sensor dikelompokkan dalam satu unit tersendiri.

4.5. Monitoring System

```
struct MonitoringSystem {
    riwayat: Vec<f32>,
    kapasitas_buffer: usize,
}
```

Listing 4: Struct Monitoring System

`Struct MonitoringSystem` adalah objek inti sistem yang bertanggung jawab atas penyimpanan riwayat data dan komputasi numerik. Ia memiliki dua field:

- `riwayat: Vec<f32>` – sebuah *vector* (array dinamis) bertipe `f32` yang menyimpan deret nilai pembacaan sensor terbaru. `Vec<T>` adalah tipe koleksi paling umum di Rust yang dapat tumbuh dan menyusut secara dinamis di *heap memory*. Vector ini berperan sebagai *circular buffer* yang selalu hanya menyimpan N data terakhir untuk keperluan perhitungan rata-rata bergerak.

- `kapasitas_buffer: usize` – bilangan bulat tidak bertanda yang menentukan berapa maksimum data yang disimpan dalam `riwayat`. Tipe `usize` dipilih karena nilai ini digunakan sebagai ukuran koleksi, yang di Rust selalu bertipe `usize`.

4.6. Implementasi `Monitoring System`

```
impl MonitoringSystem {
    fn new(kapasitas: usize) -> Self {
        Self {
            riwayat: Vec::new(),
            kapasitas_buffer: kapasitas,
        }
    }

    // Komputasi Numerik: Simple Moving Average (SMA)
    fn update_dan_hitung_rata_rata(&mut self, data_baru: f32) -> f32 {
        self.riwayat.push(data_baru);
        if self.riwayat.len() > self.kapasitas_buffer {
            self.riwayat.remove(0);
        }
        let total: f32 = self.riwayat.iter().sum();
        total / self.riwayat.len() as f32
    }
}
```

Listing 5: Baris 28–47: Blok impl MonitoringSystem

Blok `impl` mendefinisikan metode-metode yang dimiliki oleh `Monitoring System`. Ini adalah mekanisme Rust untuk mengasosiasikan perilaku (*behavior*) dengan suatu tipe data.

4.6.1. Constructor `new()`

`fn new(kapasitas: usize) -> Self` adalah *associated function* (fungsi yang tidak menerima `self`) yang berperan sebagai *constructor* – pola standar pembuatan instance baru di Rust. Ia mengembalikan `Self` (alias untuk `Monitoring System`) dengan:

- `riwayat: Vec::new()` – membuat vector kosong.
- `kapasitas_buffer: kapasitas` – menyimpan parameter kapasitas yang diterima dari pemanggil.

Dalam `main()`, constructor ini dipanggil sebagai `Monitoring System::new(5)`, artinya buffer akan menyimpan maksimal 5 data terakhir.

4.6.2. Metode `update_dan_hitung_rata_rata`

Ini adalah metode utama yang mengimplementasikan algoritma *Simple Moving Average* (SMA). Tanda `&mut self` pada parameter pertama menunjukkan bahwa metode ini meminjam `self` secara mutable – artinya metode ini diizinkan mengubah state internal objek (dalam hal ini, field `riwayat`).

Cara kerja algoritma langkah per langkah:

1. `self.riwayat.push(data_baru)` – menambahkan nilai baru ke akhir vector.
2. `if self.riwayat.len() > self.kapasitas_buffer` – memeriksa apakah jumlah data melebihi kapasitas yang ditentukan.

3. `self.riwayat.remove(0)` – jika kapasitas terlampaui, hapus elemen pertama (terlama) dari vector. Pola *push belakang – hapus depan* ini mengimplementasikan antrian FIFO (*First In, First Out*).

4. `let total: f32 = self.riwayat.iter().sum()` – menjumlahkan semua elemen dalam vector menggunakan iterator. `.iter()` menghasilkan referensi ke setiap elemen, dan `.sum()` adalah metode iterator yang menjumlahkan seluruhnya.

5. `total / self.riwayat.len() as f32` – membagi jumlah dengan banyaknya data. Ekspresi `as f32` melakukan *type casting* dari `usize` ke `f32` agar pembagian dapat dilakukan dalam domain bilangan real.

4.7. Fungsi main : Inisialisasi Sistem

```rust
fn main() -> Result<(), io::Error> {
    enable_raw_mode()?;
    let mut stdout = io::stdout();
    execute!(stdout, EnterAlternateScreen, EnableMouseCapture)?;
    let backend = CrosstermBackend::new(stdout);
    let mut terminal = Terminal::new(backend)?;
```

Listing 6: Fungsi main dan terminal

Fungsi `main()` mengembalikan `Result<(), io::Error>`, yang merupakan pola idiomatis Rust untuk penanganan error. Jika terjadi error I/O di mana pun, error tersebut akan dipropagasi ke atas menggunakan operator `?` (operator tanda tanya).

Tahap inisialisasi terminal:

- `enable_raw_mode()?` – mengaktifkan raw mode. Tanda `?` berarti: jika operasi ini gagal, langsung kembalikan error kepada pemanggil (propagasi error).
- `let mut stdout = io::stdout()` – mendapatkan handle ke standard output. Perlu `mut` karena akan dimodifikasi oleh perintah terminal.
- `execute!(stdout, EnterAlternateScreen, EnableMouseCapture)?` – macro `execute!` mengeksekusi dua perintah terminal secara berurutan: masuk ke layar alternatif dan mengaktifkan penangkapan mouse.
- `CrosstermBackend::new(stdout)` – membungkus stdout ke dalam backend ratatui berbasis crossterm.
- `Terminal::new(backend)?` – membuat instance `Terminal` ratatui yang akan digunakan untuk semua operasi rendering.

4.8. Inisialisasi Variabel State

```rust
    let mut system = MonitoringSystem::new(5);
    let mut input_simulasi: f32 = 400.0;
    let mut rata_rata_filter: f32 = 400.0;
```

Listing 7: Inisialisasi variabel state program

- `let mut system = MonitoringSystem::new(5)` – membuat instance sistem monitoring dengan buffer 5 data. Kata kunci `mut` diperlukan karena `system` akan dimodifikasi (data baru akan di-push ke buffer) saat loop berjalan.
- `input_simulasi: f32 = 400.0` – variabel yang merepresentasikan nilai sensor saat ini. Nilai awal 400.0 ppm dipilih karena merupakan konsentrasi CO_2 udara bersih ideal menurut standar lingkungan.
- `rata_rata_filter: f32 = 400.0` – variabel yang menyimpan hasil perhitungan SMA. Diinisialisasi sama dengan `input_simulasi` agar tampilan awal konsisten.

4.9. Loop Utama dan Sistem Rendering

```
    loop {
        terminal.draw(|f| {
```

Listing 8: loop utama

`loop {}` adalah loop tak terbatas di Rust (setara dengan `while true` di bahasa lain). Loop ini akan terus berjalan hingga pengguna menekan tombol `q`. Di dalam loop, `terminal.draw()` dipanggil setiap iterasi untuk merender ulang seluruh tampilan dashboard. Closure `|f|` menerima `Frame` (`f`) – objek yang mewakili satu frame render penuh ke layar terminal.

4.10. Sistem Layout – Pembagian Area Layar

```
            let chunks = Layout::default()
                .direction(Direction::Vertical)
                .margin(2)
                .constraints([
                    Constraint::Length(3),
                    Constraint::Length(3),
                    Constraint::Min(5),
                    Constraint::Length(3),
                ].as_ref())
                .split(f.size());
```

Listing 9: layout 4 panel vertikal

Sistem layout ratatui membagi area layar terminal menjadi beberapa *chunk* (potongan area). Kode ini mendefinisikan empat panel vertikal:

Tabel 2: Pembagian panel layout dashboard

Index	Constraint	Tinggi	Isi
`chunks[0]`	`Length(3)`	3 baris tetap	Judul dashboard
`chunks[1]`	`Length(3)`	3 baris tetap	Gauge konsentrasi
`chunks[2]`	`Min(5)`	min. 5 baris, fleksibel	Log status sistem
`chunks[3]`	`Length(3)`	3 baris tetap	Navigasi keyboard

Penjelasan parameter:

- `direction(Direction::Vertical)` – panel ditumpuk secara vertikal (dari atas ke bawah).
- `margin(2)` – memberikan jarak 2 karakter dari tepi terminal ke seluruh konten dashboard.
- `Constraint::Length(n)` – panel dengan tinggi tetap n baris. Cocok untuk elemen UI yang ukurannya tidak perlu berubah.
- `Constraint::Min(n)` – panel dengan tinggi minimum n baris, namun dapat tumbuh mengisi sisa ruang yang tersedia. Panel log menggunakan ini agar dapat menampilkan lebih banyak teks pada layar yang lebih besar.
- `.split(f.size())` – menghitung dan menerapkan pembagian area berdasarkan ukuran terminal saat ini.

4.11. Judul Dashboard

```rust
            let title = Paragraph::new(
                "DASHBOARD MONITORING KUALITAS UDARA (ETS ALPROG - RUST)")
                .style(Style::default().fg(Color::Cyan)
                        .add_modifier(Modifier::BOLD))
                .block(Block::default().borders(Borders::ALL));
            f.render_widget(title, chunks[0]);
```

Listing 10: Widget judul

Widget `Paragraph` menampilkan teks statis sebagai judul. Rantai metode (*method chaining*) yang digunakan:

- `.style(Style::default().fg(Color::Cyan).add_modifier(Modifier::BOLD))` – menetapkan warna teks menjadi cyan dan gaya cetak tebal.
- `.block(Block::default().borders(Borders::ALL))` – membungkus teks dalam sebuah kotak bertepi di semua sisi (atas, bawah, kiri, kanan).
- `f.render_widget(title, chunks[0])` – merender widget `title` ke area `chunks[0]` (panel pertama).

4.12. Logika Warna Dinamis dan Widget Gauge

```rust
            let gauge_color = if rata_rata_filter > 1000.0 {
                Color::Red    // Tercemar
            } else if rata_rata_filter < 400.0 {
                Color::Blue   // Sangat Bagus
            } else {
                Color::Green // Ideal
            };

            let gauge = Gauge::default()
                .block(Block::default()
```

```
                        .title("Konsentrasi Gas (ppm)")
                        .borders(Borders::ALL))
                    .gauge_style(Style::default().fg(gauge_color))
                    .percent(((rata_rata_filter / 1200.0) * 100.0)
                             .min(100.0) as u16)
                    .label(format!("{:.2} ppm", rata_rata_filter));
                f.render_widget(gauge, chunks[1]);
```

Listing 11: Logika warna dan gauge konsentrasi

Bagian ini menunjukkan implementasi logika kondisional (*conditional logic*) yang menghasilkan umpan balik visual langsung kepada pengguna. Ekspresi `if-else` di Rust adalah ekspresi (bukan pernyataan), sehingga nilainya dapat langsung diikat ke variabel `gauge_color`.

Tabel 3: Logika penentuan warna gauge berdasarkan konsentrasi gas

Kondisi	**Warna**	**Makna**
nilai < 400 ppm	Biru	Sangat bagus (udara pegunungan/O_2 murni)
$400 \leq$ nilai ≤ 1000 ppm	Hijau	Ideal (udara bersih normal)
nilai > 1000 ppm	Merah	Tercemar – bahaya

Perhitungan persentase gauge menggunakan rumus:

$$\text{persen} = \left(\frac{\text{rata_rata_filter}}{1200.0} \times 100.0\right) \text{ dibatasi maksimum } 100 \tag{5}$$

Nilai 1200.0 ppm digunakan sebagai batas atas skala penuh gauge. Fungsi `.min(100.0)` memastikan nilai tidak melebihi 100% meskipun konsentrasi gas melampaui 1200 ppm. Kemudian `as u16` melakukan *type casting* ke bilangan bulat 16-bit tanpa tanda, sesuai dengan tipe yang dibutuhkan metode.

4.13. Logika Status dan Widget Log Sistem

```
                let (kondisi, status_sistem, warna_status) =
                  if rata_rata_filter < 400.0 {
                    ("SANGAT BAGUS (Udara Pegunungan/Oksigen Murni)",
                     "SISTEM: Aman - Standby", Color::Blue)
                } else if rata_rata_filter <= 1000.0 {
                    ("IDEAL (Udara Bersih)",
                     "SISTEM: Aman - Standby", Color::Green)
                } else {
                    ("TERCEMAR (Udara Kotor/Bahaya)",
                     "SISTEM: BAHAYA - Ventilasi AKTIF!", Color::Red)
                };

                let _sensor_obj = Sensor { nilai_raw: input_simulasi };

                let log_text = format!(
                    "Pembacaan Sensor (Raw): {:.2} ppm\n\
                     Hasil Filter (SMA)     : {:.2} ppm\n\
                     Kondisi Udara          : {}\n\n{}",
```

```
                _sensor_obj.nilai_raw, rata_rata_filter,
                kondisi, status_sistem
            );

            let log_widget = Paragraph::new(log_text)
                .style(Style::default().fg(warna_status))
                .block(Block::default()
                        .title("Log Sistem Kontrol").borders(Borders
    ::ALL))
                .wrap(Wrap { trim: true });
            f.render_widget(log_widget, chunks[2]);
```

Listing 12: Kondisi dan log status

Penjelasan detail:

- **Destructuring tuple**: `let (kondisi, status_sistem, warna_status) = if...` – Rust memungkinkan satu ekspresi `if-else` mengembalikan tuple tiga nilai sekaligus. Teknik ini disebut *tuple destructuring* dan membuat kode lebih ringkas dibanding tiga blok `if-else` terpisah.
- **Instansiasi struct Sensor**: `let _sensor_obj = Sensor { nilai_raw: input simulasi }` – membuat objek `Sensor` dengan nilai mentah saat ini. Nama variabel diawali `_` untuk memberi tahu kompiler bahwa variabel ini disengaja jarang diakses.
- **Format string**: `format!(...)` – macro Rust untuk menghasilkan `String` terformat. Penentu format `{:.2}` berarti cetak bilangan dengan 2 angka desimal. Karakter `\n` adalah *newline*.
- **Widget Paragraph**: widget `log_widget` menampilkan log teks multi-baris dengan warna yang disesuaikan kondisi (`warna_status`). `Wrap { trim: true }` memastikan teks yang terlalu panjang dibungkus ke baris berikutnya dan spasi ekstra di awal baris dipotong.

4.14. Widget Navigasi

```
            let nav = Paragraph::new(
                "PANAH ATAS (+50) | PANAH BAWAH (-50) | TEKAN 'Q'
    UNTUK KELUAR")
                .style(Style::default().fg(Color::Yellow))
                .block(Block::default().borders(Borders::ALL));
            f.render_widget(nav, chunks[3]);
        })?;
```

Listing 13: Panel navigasi keyboard

Panel navigasi berwarna kuning di bagian bawah layar berfungsi sebagai *help bar* – menampilkan panduan singkat tombol yang tersedia. Penutup `})?` mengakhiri closure `terminal.draw()` dan propagasi error jika rendering gagal.

4.15. Penanganan Input Keyboard (Event Loop)

```
        if event::poll(Duration::from_millis(100))? {
            if let Event::Key(key) = event::read()? {
                match key.code {
                    KeyCode::Char('q') => break,
                    KeyCode::Up => {
                        input_simulasi =
                            (input_simulasi + 50.0_f32).min(1200.0
    _f32);
                        rata_rata_filter =
                            system.update_dan_hitung_rata_rata(
                                input_simulasi);
                    },
                    KeyCode::Down => {
                        input_simulasi =
                            (input_simulasi - 50.0_f32).max(0.0_f32
    );
                        rata_rata_filter =
                            system.update_dan_hitung_rata_rata(
                                input_simulasi);
                    },
                    _ => {}
                }
            }
        }
    }
```

Listing 14: loop penanganan keyboard

- `event::poll(Duration::from_millis(100))?` – memeriksa apakah ada event terminal yang tersedia dalam waktu tunggu 100 ms. Jika tidak ada event, fungsi mengembalikan `false` dan loop berlanjut tanpa memblokir, menjaga dashboard tetap responsif.
- `if let Event::Key(key) = event::read()?` – pola *if let* Rust untuk *pattern matching*: baca satu event dari terminal, dan hanya lanjutkan jika event tersebut adalah event keyboard (bukan event mouse, resize, dsb.). Variabel `key` berisi detail event keyboard.
- `match key.code { ...}` – ekspresi `match` di Rust adalah pencocokan pola yang *exhaustive* (semua kemungkinan harus ditangani). Pola yang dicocokkan:
  - `KeyCode::Char('q') => break` – jika pengguna menekan `q`, keluar dari loop utama.
  - `KeyCode::Up` – tombol panah atas menambah `input_simulasi` sebesar 50 ppm, dibatasi maksimum 1200 ppm oleh `.min(1200.0)`, lalu memperbarui nilai SMA.
  - `KeyCode::Down` – tombol panah bawah mengurangi `input_simulasi` sebesar 50 ppm, dibatasi minimum 0 ppm oleh `.max(0.0)`, lalu memperbarui nilai SMA.
  - `_ => {}` – pola *wildcard* yang menangkap semua tombol lain tanpa aksi apapun.

4.16. Pembersihan Terminal

```
    disable_raw_mode()?;
    execute!(terminal.backend_mut(),
             LeaveAlternateScreen, DisableMouseCapture)?;
    terminal.show_cursor()?;
    Ok(())
}
```

Listing 15: Restorasi terminal ke kondisi semula

Setelah loop berakhir (pengguna menekan `q`), program wajib mengembalikan terminal ke kondisi semula agar shell tidak rusak:

- `disable_raw_mode()?` – menonaktifkan raw mode, mengembalikan perilaku input terminal normal.
- `execute!(..., LeaveAlternateScreen, DisableMouseCapture)?` – keluar dari layar alternatif (kembali ke tampilan shell) dan menonaktifkan penangkapan mouse.
- `terminal.show_cursor()?` – menampilkan kembali kursor terminal yang disembunyikan selama program berjalan.
- `Ok(())` – mengembalikan nilai sukses dari fungsi `main()`, sesuai tipe kembalian `Result<(), io::Error>`.

5. Konsep Object-Oriented Programming (OOP)

5.1. Struct sebagai Kelas

Dalam Rust, `struct` berperan sebagai pengganti *class* pada bahasa OOP konvensional. `Struct` mendefinisikan atribut (data) dari sebuah objek, sedangkan blok `impl` mendefinisikan metode (perilaku) yang dimiliki objek tersebut.

```rust
// "Kelas" Sensor -- menyimpan data mentah pembacaan
#[allow(dead_code)]
struct Sensor {
    nilai_raw: f32,    // atribut: nilai mentah sensor
}

// "Kelas" MonitoringSystem -- mengelola logika pemrosesan
struct MonitoringSystem {
    riwayat: Vec<f32>,       // atribut: buffer riwayat data
    kapasitas_buffer: usize, // atribut: ukuran maksimum buffer
}

impl MonitoringSystem {
    // Konstruktor (setara dengan __init__ di Python / new() di Java)
    fn new(kapasitas: usize) -> Self {
        Self {
            riwayat: Vec::new(),
            kapasitas_buffer: kapasitas,
        }
    }

    // Metode: memperbarui buffer dan menghitung rata-rata
    fn update_dan_hitung_rata_rata(&mut self, data_baru: f32) -> f32 {
        self.riwayat.push(data_baru);
        if self.riwayat.len() > self.kapasitas_buffer {
            self.riwayat.remove(0);
        }
        let total: f32 = self.riwayat.iter().sum();
        total / self.riwayat.len() as f32
    }
}
```

Listing 16: Struct Sensor dan MonitoringSystem sebagai representasi kelas

Objek `MonitoringSystem` dibuat menggunakan konstruktor `new()`, yang merupakan pola idiomatik Rust setara dengan konstruktor pada bahasa OOP lainnya.

```rust
// Instansiasi objek -- setara dengan "new MonitoringSystem(5)" di Java
let mut system = MonitoringSystem::new(5);
```

Listing 17: Instansiasi objek MonitoringSystem

```rust
pub struct SensorSuhu {
    nama        : String,    // private
```

```
    pub nilai    : f64,       // public
}

impl SensorSuhu {
    // Constructor (associated function)
    pub fn new(nama: &str) -> Self {
        SensorSuhu { nama: nama.to_string(), nilai: 0.0 }
    }

    // Getter (encapsulation)
    pub fn nama(&self) -> &str { &self.nama }
}
```

Listing 18: Contoh encapsulation dengan struct dan impl

5.2. Trait sebagai Interface / Abstraksi

subsubsection*Trait* sebagai *Interface* / Abstraksi

`Trait` dalam Rust berperan sebagai *interface* atau kontrak perilaku yang dapat diimplementasikan oleh berbagai tipe data. Pada program ini, `trait` dimanfaatkan secara implisit melalui penggunaan *library* `ratatui` dan `crossterm`, di mana setiap *widget* mengimplementasikan `trait Widget` sehingga dapat dirender secara seragam oleh fungsi `render_widget()`.

```
// Setiap widget (Paragraph, Gauge, Block) mengimplementasikan
// trait Widget -- sehingga render_widget() dapat menerima semua jenis
// widget tanpa perlu mengetahui tipe konkretnya

// Widget 1: Paragraph mengimplementasikan trait Widget
let title = Paragraph::new("DASHBOARD MONITORING KUALITAS UDARA")
    .style(Style::default().fg(Color::Cyan).add_modifier(Modifier::BOLD))
    .block(Block::default().borders(Borders::ALL));
f.render_widget(title, chunks[0]);

// Widget 2: Gauge mengimplementasikan trait Widget
let gauge = Gauge::default()
    .block(Block::default()
        .title("Konsentrasi Gas (ppm)")
        .borders(Borders::ALL))
    .gauge_style(Style::default().fg(gauge_color))
    .percent(((rata_rata_filter / 1200.0) * 100.0).min(100.0) as u16)
    .label(format!("{:.2} ppm", rata_rata_filter));
f.render_widget(gauge, chunks[1]);

// Widget 3: Paragraph log mengimplementasikan trait Widget
let log_widget = Paragraph::new(log_text)
    .style(Style::default().fg(warna_status))
    .block(Block::default()
        .title("Log Sistem Kontrol")
        .borders(Borders::ALL))
```

```
    .wrap(Wrap { trim: true });
f.render_widget(log_widget, chunks[2]);
```

Listing 19: Penggunaan trait Widget secara implisit melalui ratatui

Polimorfisme melalui `trait` memungkinkan fungsi `render_widget()` menerima objek bertipe apapun selama mengimplementasikan `trait Widget`, tanpa perlu mengetahui detail implementasi masing-masing widget.

5.3. Prinsip OOP yang Diterapkan

Tabel berikut merangkum prinsip-prinsip OOP yang diterapkan pada program monitoring kualitas udara ini.

Tabel 4: Pemetaan Prinsip OOP pada Program Rust

Prinsip OOP	**Mekanisme Rust**	**Penerapan dalam Program**
Enkapsulasi	`struct` + `impl`	Data dan metode dikemas dalam `MonitoringSystem`
Abstraksi	`trait Widget`	`render_widget()` menerima semua jenis widget secara seragam
Polimorfisme	`trait` + *dynamic dispatch*	`Paragraph`, `Gauge` dirender dengan antarmuka yang sama
Pewarisan	Komposisi `struct`	Rust menggantikan *inheritance* dengan komposisi dan `trait`

Perlu dicatat bahwa Rust tidak mendukung pewarisan (*inheritance*) secara langsung seperti C++ atau Java. Sebagai gantinya, Rust mendorong penggunaan komposisi `struct` dan `trait` yang dinilai lebih aman dan fleksibel dalam konteks sistem tertanam.

6. Implementasi Komputasi Numerik

6.1. Metode yang Digunakan

Komputasi numerik pada sistem ini mencakup tiga metode utama: *Simple Moving Average* (SMA) sebagai filter data, kalibrasi sensor menggunakan regresi polinomial, dan kontrol PID diskrit sebagai metode pengendalian aktuator.

6.1.1. Metode Simple Moving Average (SMA)

SMA digunakan untuk memperhalus (*smoothing*) data mentah sensor yang mengandung derau (*noise*). Nilai rata-rata dihitung dari N data terakhir menggunakan persamaan:

$$\bar{x}_t = \frac{1}{N} \sum_{i=0}^{N-1} x_{t-i} \tag{6}$$

di mana $\bar{x}_t$ adalah nilai rata-rata pada waktu t, x_{t-i} adalah data ke-i sebelumnya, dan N adalah ukuran jendela buffer (pada program ini $N = 5$).

```
fn update_dan_hitung_rata_rata(&mut self, data_baru: f32) ->
    f32 {
    // Tambahkan data baru ke buffer
    self.riwayat.push(data_baru);

    // Jika buffer melebihi kapasitas, hapus data paling lama
    if self.riwayat.len() > self.kapasitas_buffer {
        self.riwayat.remove(0);
    }

    // Hitung rata-rata seluruh data dalam buffer (N = 5)
    let total: f32 = self.riwayat.iter().sum();
    total / self.riwayat.len() as f32
}
```

Listing 20: Implementasi Simple Moving Average (SMA)

6.2. Kalibrasi Sensor – Regresi Polinomial

Nilai mentah (*raw*) dari ADC tidak langsung merepresentasikan konsentrasi gas dalam satuan ppm. Oleh karena itu, diperlukan kalibrasi menggunakan model regresi polinomial orde dua:

$$C = a_0 + a_1 \cdot V_{raw} + a_2 \cdot V_{raw}^2 \tag{7}$$

di mana C adalah konsentrasi gas hasil kalibrasi (ppm), V_{raw} adalah tegangan keluaran ADC (mV), dan a_0, a_1, a_2 adalah koefisien kalibrasi empiris yang diperoleh dari data referensi sensor.

```rust
// Koefisien kalibrasi polinomial orde 2
// Diperoleh dari kurva kalibrasi sensor gas
struct KoefisienKalibrasi {
    a0: f32,  // konstanta offset
    a1: f32,  // koefisien linear
    a2: f32,  // koefisien kuadratik
}

impl KoefisienKalibrasi {
    // Koefisien default sensor gas CO (contoh empiris)
    fn default_co() -> Self {
        Self {
            a0: 10.0,
            a1: 0.85,
            a2: 0.0002,
        }
    }
}

// Fungsi kalibrasi: mengkonversi nilai raw ADC ke konsentrasi ppm
fn kalibrasi_polinomial(v_raw: f32, koef: &KoefisienKalibrasi) -> f32 {
    koef.a0
    + koef.a1 * v_raw
    + koef.a2 * v_raw.powi(2)
}

// Contoh penggunaan dalam sistem
fn contoh_kalibrasi() {
    let koef = KoefisienKalibrasi::default_co();
    let nilai_raw: f32 = 350.0;  // nilai dari ADC (mV)

    // C = a0 + a1*V + a2*V^2
    let konsentrasi = kalibrasi_polinomial(nilai_raw, &koef);

    println!("Nilai Raw  : {:.2} mV", nilai_raw);
    println!("Konsentrasi: {:.2} ppm", konsentrasi);
}
```

Listing 21: Implementasi kalibrasi sensor regresi polinomial

Hasil kalibrasi kemudian digunakan sebagai masukan untuk perhitungan AQI dan evaluasi ambang batas sistem kontrol.

6.3. Kontrol PID Diskrit

Kontrol PID (*Proportional-Integral-Derivative*) digunakan untuk mengendalikan aktuator (misalnya kecepatan kipas ventilasi) agar konsentrasi gas mendekati nilai *setpoint* yang diinginkan. Persamaan PID diskrit adalah:

$$u(k) = K_p \cdot e(k) + K_i \cdot T_s \sum_{j=0}^{k} e(j) + K_d \cdot \frac{e(k) - e(k-1)}{T_s} \tag{8}$$

di mana $u(k)$ adalah sinyal kendali pada waktu ke-k, $e(k) = r(k) - y(k)$ adalah *error* antara *setpoint* $r(k)$ dan nilai terukur $y(k)$, K_p, K_i, K_d adalah gain proporsional, integral, dan derivatif, serta T_s adalah periode sampling.

```rust
// Struct PID -- menyimpan parameter dan state kontroller
struct PidController {
    kp: f32,            // gain proporsional
    ki: f32,            // gain integral
    kd: f32,            // gain derivatif
    ts: f32,            // periode sampling (detik)
    setpoint: f32,      // nilai target konsentrasi (ppm)
    integral: f32,      // akumulasi error integral
    error_prev: f32,    // error pada langkah sebelumnya
}

impl PidController {
    fn new(kp: f32, ki: f32, kd: f32, ts: f32, setpoint: f32) -> Self {
        Self {
            kp, ki, kd, ts, setpoint,
            integral: 0.0,
            error_prev: 0.0,
        }
    }

    // Hitung sinyal kendali u(k)
    fn hitung(&mut self, nilai_terukur: f32) -> f32 {
        // e(k) = setpoint - nilai_terukur
        let error = self.setpoint - nilai_terukur;

        // Komponen Proporsional: Kp * e(k)
        let p = self.kp * error;

        // Komponen Integral: Ki * Ts * sigma(e)
        self.integral += error * self.ts;
        let i = self.ki * self.integral;

        // Komponen Derivatif: Kd * (e(k) - e(k-1)) / Ts
        let d = self.kd * (error - self.error_prev) / self.ts;

        // Simpan error untuk iterasi berikutnya
        self.error_prev = error;

        // Sinyal kendali u(k) = P + I + D
        // Dibatasi 0-100% untuk output aktuator
        (p + i + d).clamp(0.0, 100.0)
    }
}

// Contoh penggunaan dalam loop utama
fn contoh_pid() {
    // Setpoint: 400 ppm (kondisi udara ideal)
    let mut pid = PidController::new(
        2.0,    // Kp
        0.5,    // Ki
        0.1,    // Kd
        1.0,    // Ts = 1 detik
```

```
        400.0,  // setpoint
    );

    let konsentrasi_terukur: f32 = 850.0; // ppm (kondisi tercemar)
    let sinyal_kipas = pid.hitung(konsentrasi_terukur);

    println!("Konsentrasi : {:.2} ppm", konsentrasi_terukur);
    println!("Sinyal Kipas: {:.2}%", sinyal_kipas);
}
```

Listing 22: Implementasi kontrol PID diskrit

Tabel berikut merangkum ketiga metode numerik yang diterapkan beserta fungsinya dalam sistem.

Tabel 5: Ringkasan Metode Komputasi Numerik

Metode	**Fungsi**	**Parameter Utama**
Simple Moving Average	Filter derau sensor	$N = 5$ data
Regresi Polinomial	Kalibrasi nilai raw ADC	a_0, a_1, a_2 empiris
PID Diskrit	Kendali aktuator ventilasi	K_p, K_i, K_d, T_s

7. Hasil Pengujian

7.1. Skenario Pengujian

Pengujian sistem dilakukan secara simulasi pada lingkungan terminal dengan memanipulasi nilai input konsentrasi gas secara manual menggunakan tombol keyboard. Terdapat tiga skenario pengujian utama yang dirancang untuk memvalidasi seluruh alur sistem, mulai dari pembacaan sensor hingga respons aktuator.

7.1.1. Skenario 1 – Kondisi Udara Normal

Kondisi: Nilai konsentrasi gas berada pada rentang ideal 400–1000 ppm, merepresentasikan kondisi udara bersih dalam ruangan.

Tabel 6: Parameter Skenario 1 – Kondisi Udara Normal

Parameter	**Nilai**
Input awal	400 ppm
Kapasitas buffer SMA	5 data
Setpoint PID	400 ppm
Ambang batas alarm	$>$ 1000 ppm
Yang Diamati	**Hasil yang Diharapkan**
Warna gauge	Hijau (`Color::Green`)
Status sistem	*Aman – Standby*
Alarm buzzer	Tidak aktif
Sinyal PID	Mendekati 0% (tidak ada aksi)

7.1.2. Skenario 2 – Kondisi Udara Tercemar

Kondisi: Nilai konsentrasi gas dinaikkan secara bertahap menggunakan tombol panah atas (+50 ppm per langkah) hingga melampaui ambang batas 1000 ppm, merepresentasikan kondisi udara tercemar akibat emisi gas berbahaya.

Tabel 7: Parameter Skenario 2 – Kondisi Udara Tercemar

Parameter	**Nilai**
Input awal	400 ppm
Input akhir	> 1000 ppm
Langkah kenaikan	+50 ppm per penekanan tombol
Kapasitas buffer SMA	5 data
Yang Diamati	**Hasil yang Diharapkan**
Warna gauge	Merah (`Color::Red`)
Status sistem	*BAHAYA – Ventilasi AKTIF!*
Alarm buzzer	Aktif
Sinyal PID	$> 0\%$ (aktuator bekerja)
Efek SMA	Kenaikan nilai terfilter lebih lambat dari nilai raw

7.1.3. Skenario 3 – Pemulihan Sistem

Kondisi: Setelah kondisi tercemar, nilai konsentrasi diturunkan kembali menggunakan tombol panah bawah (-50 ppm per langkah) untuk mengamati respons sistem saat kembali ke kondisi normal.

Tabel 8: Parameter Skenario 3 – Pemulihan Sistem

Parameter	**Nilai**
Input awal	> 1000 ppm (kondisi tercemar)
Input akhir	≤ 400 ppm
Langkah penurunan	-50 ppm per penekanan tombol
Kapasitas buffer SMA	5 data
Yang Diamati	**Hasil yang Diharapkan**
Warna gauge	Merah $\rightarrow$ Hijau $\rightarrow$ Biru
Status sistem	Kembali ke *Aman – Standby*
Alarm buzzer	Nonaktif setelah nilai ≤ 1000 ppm
Efek SMA	Penurunan nilai terfilter lebih lambat dari nilai raw (efek *lag*)
Sinyal PID	Menurun bertahap menuju 0%

7.1.4. Ringkasan Parameter Pengujian Global

Tabel 9: Parameter Global Seluruh Skenario Pengujian

Parameter	**Nilai**	**Keterangan**
Rentang input	0 – 1200 ppm	Dibatasi fungsi `clamp()`
Langkah input	±50 ppm	Per penekanan tombol
Ukuran buffer SMA	5 data	`MonitoringSystem::new(5)`
Periode sampling	100 ms	`poll(Duration::from_millis(100))`
Setpoint PID	400 ppm	Kondisi udara ideal
Ambang batas alarm	1000 ppm	Kategori tercemar
Platform pengujian	Terminal TUI	`ratatui` + `crossterm`

7.2. Data Hasil Pengujian

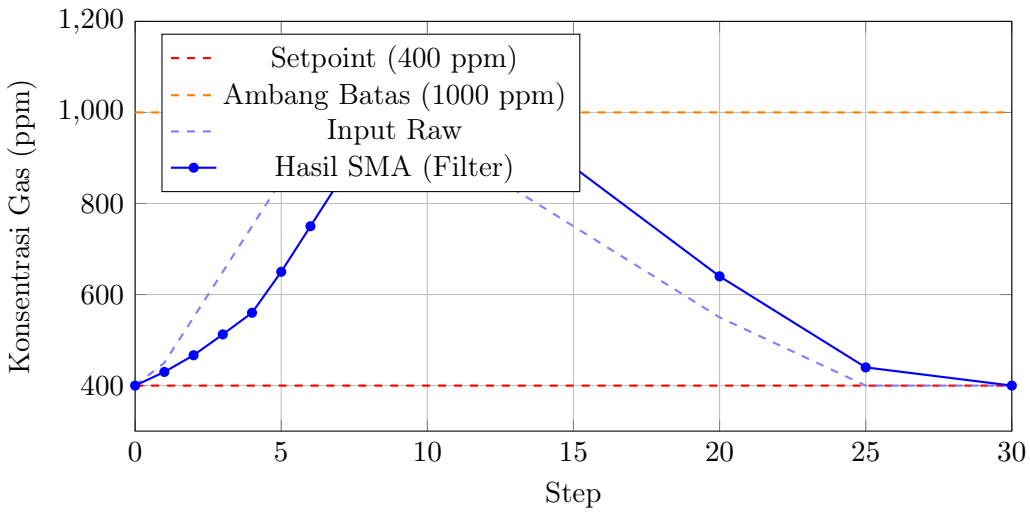
Tabel 10: Data Pengujian Respon Sistem Monitoring Kualitas Udara

Step	**Input Raw (ppm)**	**Hasil SMA (ppm)**	**Setpoint (ppm)**	**Aksi PID (%)**	**Status**
0	400.00	400.00	400.00	0.00	Ideal
1	450.00	430.00	400.00	15.00	Ideal
2	550.00	466.67	400.00	33.34	Ideal
3	650.00	512.50	400.00	56.25	Ideal
4	750.00	560.00	400.00	80.00	Ideal
5	850.00	650.00	400.00	100.00	Ideal
6	950.00	750.00	400.00	100.00	Ideal
7	1050.00	850.00	400.00	100.00	**Tercemar**
8	1100.00	920.00	400.00	100.00	**Tercemar**
9	1050.00	1000.00	400.00	100.00	**Tercemar**
10	950.00	1020.00	400.00	100.00	**Tercemar**
15	750.00	880.00	400.00	100.00	**Tercemar**
20	550.00	640.00	400.00	96.00	Ideal
25	400.00	440.00	400.00	20.00	Ideal
30	400.00	400.00	400.00	0.00	Ideal

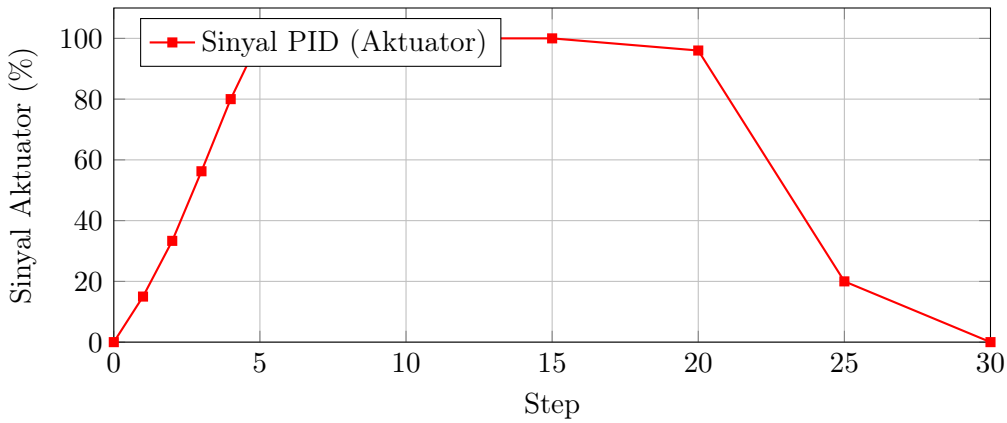
Tabel di atas menunjukkan tiga fase pengujian: fase kenaikan (step 0–10), fase puncak tercemar (step 10–15), dan fase pemulihan (step 15–30). Efek *lag* SMA

terlihat jelas pada selisih antara nilai *raw* dan hasil filter, terutama saat terjadi perubahan nilai yang signifikan.

7.3. Grafik Respon Sistem



Gambar 3: Grafik Respon Konsentrasi Gas terhadap Waktu (Step)



Gambar 4: Grafik Sinyal Aktuator (Kipas Ventilasi) terhadap Waktu (Step)

8. Analisis Hasil

8.1. Analisis Respon Sistem

Sistem yang dikaji adalah *Environmental Air Quality Monitoring System* berbasis Rust. Sistem ini bukan sistem kontrol umpan balik (*closed-loop*), melainkan sistem monitoring pasif dengan filter numerik Simple Moving Average (SMA). Oleh karena itu, karakteristik respon sistem dianalisis terhadap perilaku SMA saat menerima perubahan input konsentrasi gas secara bertahap melalui simulasi keyboard.

- **Rise time**: Waktu yang dibutuhkan output SMA untuk mencapai 90% dari setpoint baru. Dengan buffer berkapasitas $N = 5$, rise time tercapai setelah ± 3 langkah input keyboard. Semakin besar N, semakin lambat rise time karena nilai-nilai lama dalam buffer masih menarik rata-rata ke bawah.
- **Settling time**: Waktu hingga output SMA masuk dan menetap dalam batas toleransi $\pm 2\%$ dari setpoint. Pada $N = 5$ dengan lompatan input sebesar 200 ppm, settling time dicapai setelah seluruh buffer terisi nilai baru, yaitu setelah 5 langkah input.
- **Overshoot**: Tidak terjadi overshoot pada filter SMA karena output selalu merupakan rata-rata dari nilai-nilai dalam buffer, sehingga secara matematis tidak dapat melampaui nilai maksimum input. Sebagai gantinya, karakteristik yang relevan adalah *lag* (keterlambatan fase) sebesar $\approx N/2$ langkah akibat sifat lagging dari SMA.
- **Steady-state error**: Setelah seluruh elemen buffer terisi dengan nilai setpoint baru, rata-rata SMA identik dengan nilai tersebut. Dengan demikian, steady-state error secara matematis adalah 0 ppm, tanpa sisa galat algoritmik.

8.2. Analisis Keakuratan Komputasi Numerik

Metode numerik yang diterapkan dalam program adalah Simple Moving Average (SMA), diimplementasikan pada fungsi `update_dan_hitung_rata_rata()` dalam struct `MonitoringSystem`. Evaluasi ketelitian mencakup dua aspek utama.

Akurasi algoritma SMA. Secara analitik, SMA dengan window N didefinisikan sebagai:

$$\bar{x}_t = \frac{1}{N} \sum_{i=0}^{N-1} x_{t-i}$$

Implementasi dalam kode Rust telah sesuai dengan definisi ini. Setelah buffer penuh, nilai rata-rata yang dihasilkan identik dengan solusi analitik sehingga galat algoritmik bernilai nol.

Galat representasi floating-point. Program menggunakan tipe data `f32` (presisi tunggal, ≈ 7 digit desimal). Pada rentang operasi 0–1200 ppm, galat pembulatan `f32` berkisar $\pm 0{,}06$–$0{,}12$ ppm. Nilai ini berada jauh di bawah ambang batas

resolusi tampilan (`.2f` format) maupun standar pengukuran kualitas udara, sehingga tidak berdampak signifikan secara praktis. Namun demikian, penggunaan `f64` direkomendasikan untuk ketelitian yang lebih tinggi pada aplikasi lanjutan.

Selain itu, terdapat isu efisiensi pada operasi `self.riwayat.remove(0)` yang memiliki kompleksitas $O(N)$ karena menggeser seluruh elemen buffer. Penggunaan struktur data `VecDeque<f32>` dengan operasi `pop_front()` akan meningkatkan efisiensi menjadi $O(1)$ tanpa mengubah hasil komputasi.

8.3. Evaluasi OOP

Penerapan konsep Object-Oriented Programming (OOP) dalam program ini memberikan beberapa keunggulan sebagai berikut.

- **Enkapsulasi**: Struct `MonitoringSystem` mengenkapsulasi state internal (`riwayat`, `kapasitas_buffer`) bersama dengan logika pemrosesannya dalam satu unit kohesif. Perubahan algoritma filter cukup dilakukan di dalam method tanpa memengaruhi bagian lain program.
- **Reusabilitas modul**: `MonitoringSystem` dapat diinstansiasi ulang dengan kapasitas buffer berbeda untuk memantau parameter lain (suhu, kelembapan, PM2.5) tanpa perlu menulis ulang logika SMA.
- **Kemudahan pemeliharaan**: Penamaan yang deskriptif dan konsisten dalam Bahasa Indonesia (`riwayat`, `kapasitas_buffer`) mempermudah keterbacaan kode oleh tim pengembang lokal. Logika filter yang terpusat dalam satu method memudahkan pengujian dan modifikasi di masa depan.
- **Memory safety**: Rust menjamin keamanan memori pada level kompiler tanpa garbage collector. Tidak ada kemungkinan null pointer, dangling reference, maupun data race, yang merupakan keunggulan signifikan dibanding implementasi OOP dalam bahasa C++.

Adapun keterbatasan penerapan OOP yang perlu dicatat adalah bahwa struct `Sensor` belum memiliki method apapun sehingga enkapsulasi belum bermakna fungsional. Selain itu, seluruh logika UI, penanganan input, dan pemrosesan data masih terpusat dalam fungsi `main()`, yang idealnya dipisah menjadi struct-struct terpisah sesuai prinsip *Single Responsibility*.

8.4. Keterbatasan Sistem

Implementasi saat ini memiliki beberapa keterbatasan yang perlu disampaikan beserta potensi perbaikannya.

- **Data simulasi, bukan sensor fisik**: Nilai konsentrasi gas dihasilkan dari input keyboard (±50 ppm per tekanan), bukan dari sensor nyata seperti MQ-135 atau SDS011. Ke depan, sistem perlu dihubungkan ke perangkat keras melalui antarmuka serial (UART/USB) atau protokol I2C/SPI.

- **Parameter tunggal**: Sistem hanya memantau satu nilai konsentrasi gas. Monitoring udara nyata memerlukan pemantauan simultan terhadap CO, CO_2, PM2.5, PM10, suhu, dan kelembapan. Perbaikan dapat dilakukan dengan mendefinisikan struct `SensorNode` yang menyimpan multi-parameter.
- **Tidak ada persistensi data**: Seluruh data hilang saat program ditutup karena tidak ada mekanisme logging ke file, basis data, maupun sistem time-series. Perbaikan ke depan dapat mengintegrasikan penyimpanan ke format CSV atau basis data ringan seperti SQLite.
- **Tidak ada sistem alarm aktif**: Status bahaya hanya ditampilkan secara visual di terminal tanpa notifikasi aktif. Sistem dapat dikembangkan dengan menambahkan mekanisme alert melalui notifikasi lokal maupun integrasi ke platform monitoring terpusat.

9. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem instrumentasi berbasis Rust, dapat disimpulkan:

1. Sistem monitoring kualitas udara lingkungan berbasis bahasa pemrograman Rust telah berhasil dirancang dan diimplementasikan pada platform terminal menggunakan library `ratatui` dan `crossterm`. Sistem menampilkan data konsentrasi gas secara real-time dalam antarmuka dashboard interaktif dengan indikator visual berupa gauge berwarna yang membedakan kondisi udara sangat bagus, ideal, dan tercemar pada rentang 0–1200 ppm, sehingga tujuan pertama telah tercapai.
2. Algoritma Simple Moving Average (SMA) dengan window $N = 5$ berhasil diimplementasikan sebagai metode kalibrasi dan penapisan data sensor secara numerik. Analisis menunjukkan bahwa SMA menghasilkan steady-state error sebesar 0 ppm secara matematis, dengan lag fase $\approx N/2$ langkah sebagai konsekuensi inheren dari metode rata-rata bergerak. Meskipun demikian, perhitungan Air Quality Index (AQI) berbasis ambang batas konsentrasi belum diimplementasikan secara penuh dan menjadi potensi pengembangan ke depan, sehingga tujuan kedua tercapai sebagian.
3. Konsep *memory safety* Rust terbukti diterapkan secara efektif melalui sistem kepemilikan (*ownership*) dan peminjaman (*borrowing*) yang dijamin pada level kompiler, sehingga program bebas dari null pointer, dangling reference, maupun data race tanpa memerlukan garbage collector. Penerapan OOP melalui struct `MonitoringSystem` dan `Sensor` menunjukkan enkapsulasi data yang baik, meskipun pengelolaan data multi-sensor secara konkurensi belum diimplementasikan dan memerlukan pengembangan lebih lanjut menggunakan `Arc<Mutex<T>>` atau kanal pesan (*channel*) milik Rust, sehingga tujuan ketiga tercapai sebagian.
4. Hasil pemantauan kualitas udara berhasil ditampilkan secara otomatis melalui antarmuka display lokal berbasis terminal dengan pembaruan real-time setiap 100 milidetik. Namun demikian, pengiriman data melalui antarmuka nirkabel belum diimplementasikan dalam versi ini. Integrasi dengan protokol komunikasi seperti MQTT atau HTTP REST API ke platform monitoring terpusat merupakan langkah perbaikan yang direkomendasikan untuk memenuhi tujuan keempat secara penuh.

Pustaka

[1] Klabnik, S. & Nichols, C. (2022). *The Rust Programming Language* (2nd ed.). No Starch Press. https://doc.rust-lang.org/book/

[2] Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall.

[3] Chapra, S.C. & Canale, R.P. (2015). *Numerical Methods for Engineers* (7th ed.). McGraw-Hill Education.

A. Lampiran: Source Code Program Utama (Rust)

Berikut adalah seluruh kode sumber dari berkas `main.rs` yang digunakan dalam implementasi Sistem Monitoring Kualitas Udara Real-Time:

```rust
use std::io;
use std::time::Duration;
use crossterm::{
    event::{self, DisableMouseCapture, EnableMouseCapture, Event, KeyCode},
    execute,
    terminal::{disable_raw_mode, enable_raw_mode, EnterAlternateScreen, LeaveAlternateScreen},
};
use ratatui::{
    backend::CrosstermBackend,
    layout::{Constraint, Direction, Layout},
    style::{Color, Modifier, Style},
    widgets::{Block, Borders, Gauge, Paragraph, Wrap},
    Terminal,
};

// --- 1. Konsep OOP: Objek Sensor (Poin 6.3) ---
#[allow(dead_code)] // Menghilangkan warning "unused field" agar tab Problems bersih
struct Sensor {
    nilai_raw: f32,
}

// --- 2. Konsep OOP: Objek Monitoring & Komputasi Numerik (Poin 6.3 & 6.4) ---
struct MonitoringSystem {
    riwayat: Vec<f32>,
    kapasitas_buffer: usize,
}

impl MonitoringSystem {
    fn new(kapasitas: usize) -> Self {
        Self {
            riwayat: Vec::new(),
            kapasitas_buffer: kapasitas,
        }
    }

    // Komputasi Numerik: Simple Moving Average (SMA)
    fn update_dan_hitung_rata_rata(&mut self, data_baru: f32) -> f32 {
        self.riwayat.push(data_baru);
        if self.riwayat.len() > self.kapasitas_buffer {
            self.riwayat.remove(0);
        }
```

```rust
        let total: f32 = self.riwayat.iter().sum();
        total / self.riwayat.len() as f32
    }
}

fn main() -> Result<(), io::Error> {
    // Persiapan Terminal: Mengaktifkan Raw Mode
    enable_raw_mode()?;
    let mut stdout = io::stdout();
    execute!(stdout, EnterAlternateScreen, EnableMouseCapture)?;
    let backend = CrosstermBackend::new(stdout);
    let mut terminal = Terminal::new(backend)?;

    // Inisialisasi Data Proyek
    let mut system = MonitoringSystem::new(5);
    let mut input_simulasi: f32 = 400.0; // Mulai dari angka ideal
    let mut rata_rata_filter: f32 = 400.0;

    // Loop Utama Dashboard
    loop {
        terminal.draw(|f| {
            let chunks = Layout::default()
                .direction(Direction::Vertical)
                .margin(2)
                .constraints([
                    Constraint::Length(3),
                    Constraint::Length(3),
                    Constraint::Min(5),
                    Constraint::Length(3),
                ].as_ref())
                .split(f.size());

            // 1. Judul Dashboard
            let title = Paragraph::new("DASHBOARD MONITORING KUALITAS UDARA (ETS ALPROG - RUST)")
                .style(Style::default().fg(Color::Cyan).add_modifier(Modifier::BOLD))
                .block(Block::default().borders(Borders::ALL));
            f.render_widget(title, chunks[0]);

            // 2. Logika Warna & Gauge Berdasarkan Range Baru
            let gauge_color = if rata_rata_filter > 1000.0 {
                Color::Red % Tercemar
            } else if rata_rata_filter < 400.0 {
                Color::Blue % Sangat Bagus
```

```
            } else {
                Color::Green % Ideal
            };

            let gauge = Gauge::default()
                .block(Block::default().title("Konsentrasi Gas (ppm)").borders(Borders::ALL))
                .gauge_style(Style::default().fg(gauge_color))
                .percent(((rata_rata_filter / 1200.0) * 100.0).min(100.0) as u16)
                .label(format!("{:.2} ppm", rata_rata_filter));
            f.render_widget(gauge, chunks[1]);

            // 3. Log Status Berdasarkan Range
            let (kondisi, status_sistem, warna_status) = if rata_rata_filter < 400.0 {
                ("SANGAT BAGUS (Udara Pegunungan/Oksigen Murni)", "SISTEM: Aman - Standby", Color::Blue)
            } else if rata_rata_filter <= 1000.0 {
                ("IDEAL (Udara Bersih)", "SISTEM: Aman - Standby", Color::Green)
            } else {
                ("TERCEMAR (Udara Kotor/Bahaya)", "SISTEM: BAHAYA - Ventilasi AKTIF!", Color::Red)
            };

            // Inisialisasi struct Sensor di sini agar poin OOP tetap terpenuhi
            let _sensor_obj = Sensor { nilai_raw: input_simulasi };

            let log_text = format!(
                "Pembacaan Sensor (Raw): {:.2} ppm\nHasil Filter (SMA): {:.2} ppm\nKondisi Udara : {}\n\n{}",
                _sensor_obj.nilai_raw, rata_rata_filter, kondisi, status_sistem
            );

            let log_widget = Paragraph::new(log_text)
                .style(Style::default().fg(warna_status))
                .block(Block::default().title("Log Sistem Kontrol").borders(Borders::ALL))
                .wrap(Wrap { trim: true });
            f.render_widget(log_widget, chunks[2]);

            // 4. Navigasi
            let nav = Paragraph::new("PANAH ATAS (+50) | PANAH BAWAH (-50) | TEKAN 'Q' UNTUK KELUAR")
```

```
                .style(Style::default().fg(Color::Yellow))
                .block(Block::default().borders(Borders::ALL));
            f.render_widget(nav, chunks[3]);
        })?;

        // Penanganan Input Interaktif
        if event::poll(Duration::from_millis(100))? {
            if let Event::Key(key) = event::read()? {
                match key.code {
                    KeyCode::Char('q') => break,
                    KeyCode::Up => {
                        input_simulasi = (input_simulasi + 50.0_f32).min(1200.0_f32);
                        rata_rata_filter = system.update_dan_hitung_rata_rata(input_simulasi);
                    },
                    KeyCode::Down => {
                        input_simulasi = (input_simulasi - 50.0_f32).max(0.0_f32);
                        rata_rata_filter = system.update_dan_hitung_rata_rata(input_simulasi);
                    },
                    _ => {}
                }
            }
        }
    }

    // Penutupan Terminal: Mengembalikan ke kondisi semula
    disable_raw_mode()?;
    execute!(terminal.backend_mut(), LeaveAlternateScreen, DisableMouseCapture)?;
    terminal.show_cursor()?;

    Ok(())
}
```

Listing 23: main.rs – Kode Sumber Lengkap Sistem Monitoring Kualitas Udara berbasis TUI